

NAME: S.GAYATHERI

REG NO:192521306

COURSE CODE: CSA0619

COURSE NAME : Design

Analysis of Algorithm

CO4 – AT1 – CODE CHALLENGE

Q12. Longest Increasing Subsequence (LIS)

Aim

To design and implement a Dynamic Programming algorithm in Python to find the length of the Longest Increasing Subsequence (LIS) in a given sequence of integers.

Problem Statement

Given a sequence of n integers, find the length of the longest subsequence in which the elements are arranged in **strictly increasing order**.

A subsequence does not need to contain adjacent elements, but the **original order of the selected elements must be maintained**.

For example, for the sequence:

10 22 9 33 21 50

The longest increasing subsequence is:

10 $\rightarrow$ 22 $\rightarrow$ 33 $\rightarrow$ 50

Therefore, the LIS length is 4.

Objectives

- To understand the concept of subsequences.
- To apply Dynamic Programming to solve the LIS problem.
- To determine the maximum length of an increasing subsequence.
- To analyze the time and space complexity of the algorithm.
- To implement the solution using Python.

Concept of Longest Increasing Subsequence

A **subsequence** is obtained by selecting some elements from the original sequence while maintaining their relative order.

For example:

10 22 9 33 21 50

Some increasing subsequences are:

- 10 $\rightarrow$ 22
- 10 $\rightarrow$ 33
- 10 $\rightarrow$ 21 $\rightarrow$ 50
- 10 $\rightarrow$ 22 $\rightarrow$ 33 $\rightarrow$ 50

The longest one is:

10 → 22 → 33 → 50

Hence:

LIS Length = 4

The selected elements do not have to be next to each other.

Algorithm

1. Read the number of elements n.
2. Read the array elements.
3. Create a dp array of size n.
4. Initialize every value of dp to 1.
5. For every element arr[i], compare it with all previous elements.
6. If arr[i] > arr[j], update:
$$dp[i] = \max(dp[i], dp[j] + 1)$$
7. Find the maximum value in the dp array.
8. Display the maximum value as the LIS length.

Python Program

```
main.py Visualize Run
1 def longest_increasing_subsequence(arr):
2     n = len(arr)
3
4     # dp[i] stores the length of the LIS
5     # ending at index i
6     dp = [1] * n
7
8     for i in range(1, n):
9         for j in range(i):
10             if arr[i] > arr[j]:
11                 dp[i] = max(dp[i], dp[j] + 1)
12
13     return max(dp)
14
15
16 # Input
17 n = int(input("Enter number of elements: "))
18 arr = list(map(int, input("Enter the elements: ").split()))
19
20 # Find LIS
21 result = longest_increasing_subsequence(arr)
22
23 # Output
24 print("LIS Length =", result)
```

Sample Input & Output :

Output

```
Enter number of elements: 5
Enter the elements: 5 4 3 2 1
LIS Length = 1

=== Code Execution Successful ===
```

Time Complexity

The algorithm uses two nested loops.

- Outer loop runs n times.
- Inner loop can run up to n times.

Therefore, the time complexity is:

$O(n^2)$

This means the execution time increases approximately with the square of the number of elements.

Space Complexity

The algorithm uses an additional dp array of size n .

Therefore:

Space Complexity = $O(n)$

Advantages

- Simple and easy to understand.
- Uses the Dynamic Programming concept effectively.
- Works for positive, negative, and zero values.
- Maintains the original order of elements.
- Does not require the elements to be adjacent.
- Provides an exact LIS length.

Result

The Longest Increasing Subsequence problem was successfully implemented using Dynamic Programming in Python.

